Working Title Goes Here

Project Proposal

Matthew La
Bachelor of Science in Computer Science
The University of Bath

November 1, 2026

Contents

1 Problem Description **1**
1.1 Aims 1
1.2 Objectives 1

2 Requirements Specification **1**

3 Project Plan **1**
3.1 Gantt Chart 1
3.2 Tasks and Milestones 1
3.3 Risk Assessment 1

4 Resources **1**

1 Problem Description

Since the public availability of GitHub Copilot and ChatGPT in 2022, AI-assisted code generation has become a staple of software engineering, signalling a shift away from hand-written code. In 2026, Stack Overflow's Developer Survey reported that 83% of respondents used AI tools at work (Stack Overflow, 2026). Furthermore, GitHub also reports that the proportion of pull requests that are initially drafted by AI has more than doubled between March and September 2026 (GitHub, 2026).

However, studies have shown that much of AI-generated code can be incorrect, insecure, or both. For example, Vero et al. (2025) show that even OpenAI o1 may perform poorly when writing realistic software, with only 62% code correctness. Additionally, roughly half of these correct programs still contained vulnerabilities.

Trust in LLM output therefore remains a significant factor for AI adoption. Indeed, while 87% of Stack Overflow survey respondents report some level of trust in AI's outputs, 48% trust AI only when they can easily verify its output.

One of the most common methods for developers to carry out this kind of verification for their software is unit testing. Even so, test-passing AI-generated software patches still contain errors at a noticeable rate, with 29.6% of these patches behaving differently from the developers' real fix, and 28.6% of these divergences containing incorrect code (Wang, Pradel, and Liu, 2025).

More fundamentally, tests are a finite set of hand-picked inputs that inevitably miss unconsidered edge cases. As Dijkstra (1970, p. 7) observed, "program testing can be used to show the presence of bugs, but never to show their absence". Dijkstra's own later solution to this problem was to explicitly prove a program's correctness (Dijkstra, 1972).

Hoare (1969) gave this idea a formal basis, and formal verification can now guarantee that a program meets its specification for every possible input. However, these guarantees are expensive. The seL4 microkernel's roughly 8,700 lines of C required around 200,000 lines of Isabelle/HOL proof (Klein et al., 2009), and a survey of 216 practitioners identifies scalability and the expertise required as key barriers to adoption (Gleirscher and Marmsoler, 2020).

Yet the idea at the core of verification, stating explicitly what a program should do and checking the code against that statement mechanically, need not carry this cost. Design by contract (Meyer, 1992) attaches such specifications to each function as preconditions and postconditions, making intended behaviour part of the code rather than leaving it implicit in a set of tests. AI-generated code may make this more valuable still (Kleppmann, 2025): when an LLM writes the implementation, a contract records intent independently of it, giving the compiler something to check the code against.

1.1 Aims

1.2 Objectives

2 Requirements Specification

3 Project Plan

3.1 Gantt Chart

3.2 Tasks and Milestones

3.3 Risk Assessment

4 Resources

References

Dijkstra, E.W., 1970. *Notes on structured programming* [Online]. (technical report). Technological University Eindhoven. Available from: https://www.cs.utexas.edu/~EWD/ewd02xx/EWD249.PDF [Accessed November 1, 2026].

Dijkstra, E.W., 1972. The humble programmer. *Communications of the acm* [Online]. Available from: https://www.cs.utexas.edu/~EWD/transcriptions/EWD03xx/EWD340.html [Accessed November 1, 2026].

GitHub, 2026. *Octoverse: commits, pushes, reach new highs in 2026 as developers build with agents* [Online]. Available from: https://github.blog/news-insights/octoverse/octoverse-commits-pushes-reach-new-highs-in-2026-as-developers-build-with-agents/ [Accessed November 1, 2026].

Gleirscher, M. and Marmsoler, D., 2020. *Formal methods in dependable systems engineering: a survey of professionals from Europe and North America* [Online]. Available from: https://doi.org/10.1007/s10664-020-09836-5 [Accessed November 1, 2026].

Hoare, C.A.R., 1969. *An axiomatic basis for computer programming* [Online]. Available from: https://doi.org/10.1145/363235.363259 [Accessed November 1, 2026].

Klein, G., Elphinstone, K., Heiser, G., Andronick, J., Cock, D., Derrin, P., Elkaduwe, D., Engelhardt, K., Kolanski, R., Norrish, M., Sewell, T., Tuch, H., and Winwood, S., 2009. *seL4: formal verification of an OS kernel* [Online]. Available from: https://trustworthy.systems/publications/papers/Klein_EHACDEEKNSTW_09.pdf [Accessed November 1, 2026].

Kleppmann, M., 2025. *Prediction: AI will make formal verification go mainstream* [Online]. Available from: https://martin.kleppmann.com/2025/12/08/ai-formal-verification.html [Accessed November 1, 2026].

Meyer, B., 1992. *Applying "design by contract"* [Online]. Available from: https://doi.org/10.1109/2.161279 [Accessed November 1, 2026].

Stack Overflow, 2026. *2026 developer survey: AI* [Online]. Available from: https://survey.stackoverflow.co/2026/ai/data [Accessed October 31, 2026].

Vero, M., Mündler, N., Chibotaru, V., Raychev, V., Baader, M., Jovanović, N., He, J., and Vechev, M., 2025. *BaxBench: can LLMs generate correct and secure backends?* [Online]. Available from: https://baxbench.com/ [Accessed November 1, 2026].

Wang, Y., Pradel, M., and Liu, Z., 2025. *Are "solved issues" in SWE-bench really solved correctly? An empirical study* [Online]. Available from: https://arxiv.org/abs/2503.15223 [Accessed November 1, 2026].